

Unit II Notes: Markov Decision Processes (MDPs, Bellman Equations, and Dynamic Programming)

May 28, 2026

Notation

- S_t — state at time t ; A_t — action taken at time t ; R_{t+1} — reward received after taking A_t in state S_t .
- $\mathcal{S}$ — finite set of states; $\mathcal{A}$ — finite set of actions; $\gamma \in [0, 1]$ — discount factor (how much weight is placed on future rewards relative to immediate ones).
- $p(s', r \mid s, a) = \mathbb{P}[S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a]$ — the environment's transition-and-reward dynamics; $\mathcal{P}_{ss'}^a$ and $\mathcal{R}_s^a$ are the marginalised forms used in some equations.
- $\pi(a \mid s) = \mathbb{P}[A_t = a \mid S_t = s]$ — the policy (a probability distribution over actions for every state).
- $G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$ — the *return* from time t : the discounted sum of future rewards.
- $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$ — state-value function (expected return from s when following policy π); $q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$ — action-value function.
- $v_*(s) = \max_\pi v_\pi(s)$ — optimal state-value function; $q_*(s, a) = \max_\pi q_\pi(s, a)$ — optimal action-value function; π_* — an optimal policy.
- θ — a small tolerance used as a stopping condition in iterative dynamic programming.
- *Stochastic* means *random*; *deterministic* means *fixed, with no randomness*. Both words appear repeatedly in the notes.

Part A — Short Questions (2 marks each)

1. **State the Markov property and explain why it is required when defining an MDP.**

A state S_t is said to satisfy the Markov property when the distribution of the next state depends only on the present state and action and not on the earlier history, that is, $\mathbb{P}[S_{t+1} \mid S_t, A_t] = \mathbb{P}[S_{t+1} \mid S_1, A_1, \dots, S_t, A_t]$. The property is required because the value and policy iteration arguments treat the current state as a complete summary of the past; without it, value functions defined over S_t alone would be ambiguous because two trajectories ending in the same state could have different futures. The Markov property therefore guarantees that the state is a sufficient statistic for predicting future rewards.

2. **Define a Markov Decision Process by listing the components of the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$.**

An MDP is the tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$, where $\mathcal{S}$ is a finite set of states, $\mathcal{A}$ is a finite set of actions, $\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$ is the transition probability from s to s' under

action a , $\mathcal{R}_s^a = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$ is the expected immediate reward, and $\gamma \in [0, 1]$ is the discount factor that controls how much weight is placed on rewards in the distant future. Together, $\mathcal{P}$ and $\mathcal{R}$ specify the full one-step dynamics $p(s', r \mid s, a)$ used in all later equations.

- 3. Numerical — Return computation.** An agent observes the reward sequence $R_1 = 4$, $R_2 = 2$, $R_3 = 8$ over a three-step episode with discount factor $\gamma = 0.5$. Compute the returns G_2 , G_1 , and G_0 using the recursive relation $G_t = R_{t+1} + \gamma G_{t+1}$.

Working backwards from the final step, $G_2 = R_3 = 8$ because no further rewards follow. Applying the recursion,

$$G_1 = R_2 + \gamma G_2 = 2 + (0.5)(8) = 6, \quad G_0 = R_1 + \gamma G_1 = 4 + (0.5)(6) = 7.$$

The return at the start of the episode is therefore $G_0 = 7$. Computing returns recursively is preferable to applying the closed-form sum because the same expression is reused inside the Bellman equations.

- 4. Distinguish between episodic and continuing tasks, and explain the choice of γ in each case.**

An episodic task terminates at a random time T and resets afterwards (such as a game ending in a win or loss), so the return is the finite sum $G_t = R_{t+1} + R_{t+2} + \dots + R_T$ and a discount of $\gamma = 1$ is permissible. A continuing task runs indefinitely with $T = \infty$ (such as a process-control loop), so the undiscounted sum may diverge; in this case $\gamma < 1$ is required to keep the return finite. The unified return formula $G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$ covers both settings provided either $T < \infty$ or $\gamma < 1$, but not both $T = \infty$ and $\gamma = 1$ together.

- 5. Numerical — One-step Bellman expectation backup.** Under policy π , state s deterministically takes action a . The environment then transitions to s_1 with probability 0.5 (immediate reward +2) or to s_2 with probability 0.5 (immediate reward +6). Given $v_\pi(s_1) = 4$, $v_\pi(s_2) = 8$, and $\gamma = 0.5$, compute the one-step Bellman backup for $v_\pi(s)$.

The Bellman expectation backup at s is $\sum_{s',r} p(s', r \mid s, a) [r + \gamma v_\pi(s')]$. Substituting the two transitions,

$$v_\pi(s) = 0.5 [2 + (0.5)(4)] + 0.5 [6 + (0.5)(8)] = 0.5(4) + 0.5(10) = 7.$$

The backup combines the immediate rewards with discounted next-state values, weighted by the transition probabilities; a single application of this rule for every state forms one sweep of iterative policy evaluation.

- 6. Define the state-value function $v_\pi(s)$ and the action-value function $q_\pi(s, a)$ and state the relationship between them.**

The state-value function is the expected return from state s when actions are chosen according to π , namely $v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$. The action-value function is the expected return when action a is taken first in state s and π is followed thereafter, namely $q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$. The two are linked by $v_\pi(s) = \sum_a \pi(a \mid s) q_\pi(s, a)$: the state value is the action value averaged over the policy's action distribution at that state.

- 7. State the Bellman optimality equation for v_* and explain why it is non-linear.**

The Bellman optimality equation expresses the optimal state value as

$$v_*(s) = \max_a \sum_{s',r} p(s', r \mid s, a) [r + \gamma v_*(s')].$$

It is non-linear because of the max operator on the right-hand side: the function on v_* is piecewise linear but not linear, so the equation cannot be solved by a single matrix inversion in the way the linear Bellman expectation equation can. Iterative methods such as value iteration are used instead, applying the right-hand side as an update rule until the values stop changing.

- 8. Numerical — Action values and policy improvement.** From state s , two deterministic actions are available. Action a_1 moves to state s_1 with reward 2; action a_2 moves to state s_2 with reward 0. The current state values are $v_\pi(s) = 4$, $v_\pi(s_1) = 6$, $v_\pi(s_2) = 8$ with $\gamma = 0.5$. Compute $q_\pi(s, a_1)$ and $q_\pi(s, a_2)$ and identify which actions strictly improve the policy at s .

Each action is deterministic, so $q_\pi(s, a) = r + \gamma v_\pi(s')$. For a_1 , $q_\pi(s, a_1) = 2 + (0.5)(6) = 5$; for a_2 , $q_\pi(s, a_2) = 0 + (0.5)(8) = 4$. Comparing each with $v_\pi(s) = 4$, action a_1 is a strict improvement because $5 > 4$, while action a_2 only matches the current value and produces no improvement. The greedy policy at s is therefore $\pi'(s) = a_1$.

- 9. Write the iterative policy evaluation update for $V(s)$ and state the stopping criterion used in practice.**

The iterative policy evaluation update applies the Bellman expectation backup once per state per sweep:

$$V_{k+1}(s) \leftarrow \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma V_k(s')].$$

In practice the loop is terminated when the largest change $\Delta = \max_s |V_{k+1}(s) - V_k(s)|$ across all states falls below a small tolerance θ . The starting estimate V_0 is usually zero everywhere, with $V_0(\text{terminal}) = 0$ enforced throughout.

- 10. State the Policy Improvement Theorem and its consequence for the greedy policy.**

Let π and π' be two deterministic policies such that $q_\pi(s, \pi'(s)) \geq v_\pi(s)$ for every state s . The Policy Improvement Theorem states that π' is at least as good as π from every starting state, that is, $v_{\pi'}(s) \geq v_\pi(s)$ for all s ; the inequality is strict at any state where the assumption is strict. The greedy policy $\pi'(s) = \arg \max_a q_\pi(s, a)$ automatically satisfies the assumption, so building π' greedily from v_π always produces a policy that is at least as good as π .

- 11. Numerical — One step of value iteration.** State s has two available actions, both deterministic, with $\gamma = 0.5$. Action a_1 yields immediate reward 4 and lands in s' where the current estimate is $V(s') = 8$. Action a_2 yields immediate reward 10 and lands in s'' where $V(s'') = 4$. Apply one Bellman optimality backup at s and identify the corresponding greedy action.

The value iteration update is $V(s) \leftarrow \max_a [r + \gamma V(\text{next})]$. For each action,

$$r + \gamma V = 4 + (0.5)(8) = 8 \quad \text{for } a_1, \quad r + \gamma V = 10 + (0.5)(4) = 12 \quad \text{for } a_2.$$

The maximum is 12, so $V(s) \leftarrow 12$ and the greedy action is a_2 : the larger immediate reward outweighs the lower successor value at this discount.

- 12. Compare Policy Iteration and Value Iteration in terms of per-iteration cost and the number of outer iterations needed.**

Policy Iteration runs a full policy evaluation (many Bellman expectation sweeps) before a single greedy improvement, so each outer step is expensive but only a small number (typically

five to twenty) are needed to reach the optimal policy in a finite MDP. Value Iteration performs a single Bellman optimality sweep per outer step, which is cheap, but a larger number of outer steps is required because the error shrinks by a factor of γ per sweep. The total computational effort is often comparable: Policy Iteration spends time inside the evaluation loop, while Value Iteration spreads the same work across more outer iterations.

Part B — Long Questions (8 marks each)

1. **Define a Markov Decision Process and describe the agent–environment interface in full. Explain the Markov property and discuss why MDPs generalise the bandit problem.**

Definition. A Markov Decision Process is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ consisting of a finite set of states $\mathcal{S}$, a finite set of actions $\mathcal{A}$, a transition function $\mathcal{P}_{ss'}^a = \mathbb{P}[S_{t+1} = s' \mid S_t = s, A_t = a]$, an expected-reward function $\mathcal{R}_s^a = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]$, and a discount factor $\gamma \in [0, 1]$. Together $\mathcal{P}$ and $\mathcal{R}$ define the one-step dynamics $p(s', r \mid s, a)$ that drive every subsequent quantity in the theory.

Agent–environment interface. The interaction proceeds in discrete time: at each step t the agent observes the state S_t , selects an action A_t according to its policy $\pi(\cdot \mid S_t)$, and the environment responds by sampling a reward R_{t+1} and a next state S_{t+1} from $p(\cdot, \cdot \mid S_t, A_t)$. The resulting trajectory $S_0, A_0, R_1, S_1, A_1, R_2, \dots$ is the data on which the agent’s value estimates and policy updates are computed.

Markov property. A state S_t is Markov when the conditional distribution of the next state and reward depends only on (S_t, A_t) and not on the past, namely

$$\mathbb{P}[S_{t+1}, R_{t+1} \mid S_t, A_t] = \mathbb{P}[S_{t+1}, R_{t+1} \mid S_1, A_1, \dots, S_t, A_t].$$

The property states that the current state is a sufficient summary of the history for predicting the future, which is exactly what allows value functions to be defined as a function of state alone. When the property fails, the natural remedy is to redefine the state to include enough history (for example, the last k observations) until the property is restored.

From bandits to MDPs. The bandit problem of Unit I is the special case of an MDP with a single state, in which actions affect only the immediate reward and there is no notion of a successor state. An MDP generalises this in three crucial ways:

- the agent now exists in a state S_t that summarises the situation it faces;
- the chosen action influences both the immediate reward R_{t+1} and the next state S_{t+1} ;
- the objective is the cumulative future reward, not the next reward alone.

Consequences. The new structure introduces planning and credit assignment, both absent in a bandit: the agent must decide how to act now in order to reach high-value states later, and a reward received at step $t + k$ must be attributed correctly to actions taken much earlier. The bandit’s exploration–exploitation tension is still present but is now interleaved with these new concerns. Every algorithm in subsequent units operates within this formalism.

2. **Define the return G_t and explain the discounted, finite-horizon, episodic, and continuing reward formulations. Discuss why the discount factor γ is essential in the continuing case and provide a unified expression.**

Definition. The return at time t is the cumulative reward received from that point onwards.

In its most general discounted form,

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1},$$

with discount factor $\gamma \in [0, 1]$. The return obeys the recursion $G_t = R_{t+1} + \gamma G_{t+1}$, which is the building block of every later Bellman equation.

Episodic tasks. An episodic task ends at a random terminal time T and resets thereafter. Examples include playing a game until win or loss and running a maze until the exit. The return is the finite sum

$$G_t = R_{t+1} + R_{t+2} + \cdots + R_T,$$

and $\gamma = 1$ is permissible because the sum has finitely many terms. The state space is extended to $\mathcal{S}^+$ by adding an absorbing terminal state with zero reward.

Continuing tasks. A continuing task runs indefinitely; there is no natural terminal state. The undiscounted sum $R_{t+1} + R_{t+2} + \cdots$ can diverge, so the discounted form with $\gamma < 1$ is required to keep the return finite. Three reasons justify the discount factor:

- it ensures convergence whenever the rewards are bounded;
- it expresses a preference for sooner rewards over later ones, which is realistic in most engineering settings;
- it makes the resulting Bellman operator a contraction with factor γ , which is the technical property behind every dynamic programming convergence proof.

Finite-horizon and average-reward variants. In a finite-horizon formulation, the return is truncated at a fixed step T_h : $G_t = \sum_{k=0}^{T_h-t-1} R_{t+k+1}$. In the average-reward formulation, the criterion is the long-run average $\lim_{T \rightarrow \infty} \frac{1}{T} \sum_{k=1}^T R_k$, which is useful when no natural discount rate is available; this variant is mentioned only briefly in the syllabus.

Unified expression. Using the absorbing terminal trick, both episodic and continuing tasks can be written with the same formula

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1},$$

allowing $T = \infty$ (continuing) or $\gamma = 1$ (episodic with finite T) but not both simultaneously. The episodic case extends the sum past T without changing the value because every reward after termination is zero.

Example: pole-balancing. The pole-balancing task can be cast either way. In an episodic framing, an episode ends when the pole falls; each non-failure step earns reward +1 and the return is the number of steps until failure ($\gamma = 1$ is permissible). In a continuing framing, the task runs indefinitely with reward -1 on failure and 0 otherwise; here $\gamma < 1$ is required. Both lead to the same optimal behaviour — balance the pole for as long as possible — which illustrates that the choice of formulation is a modelling decision, not a property of the task itself.

3. Derive the Bellman expectation equations for $v_\pi(s)$ and $q_\pi(s, a)$. Explain their interpretation, their relationship, and what makes them solvable in closed form.

Definition. The state-value function under policy π is

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s],$$

and the action-value function is

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a].$$

Both are expectations under the trajectory distribution induced by π and the environment dynamics.

Derivation for v_π . Starting from the definition and applying the recursive return $G_t = R_{t+1} + \gamma G_{t+1}$,

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma \mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s']] \\ &= \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')]. \end{aligned}$$

The last line is the Bellman expectation equation for v_π : the value of s equals the expected immediate reward plus the discounted expected value of the next state, averaged over the policy and the dynamics.

Derivation for q_π . Conditioning instead on (S_t, A_t) , the same recursion gives

$$q_\pi(s, a) = \sum_{s', r} p(s', r \mid s, a) [r + \gamma \sum_{a'} \pi(a' \mid s') q_\pi(s', a')].$$

The bracket separates the one-step contribution r from the discounted future value, which is itself an average over the policy's choice at s' .

Relationship. The two functions are linked by two complementary identities:

- $v_\pi(s) = \sum_a \pi(a \mid s) q_\pi(s, a)$: the state value is the action-value averaged under the policy;
- $q_\pi(s, a) = \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')]$: the action value is a one-step lookahead into the state values.

Linearity and closed-form solution. In matrix form, the Bellman expectation equation for v_π reads

$$\mathbf{v}_\pi = \mathbf{r}^\pi + \gamma \mathbf{P}^\pi \mathbf{v}_\pi,$$

where $\mathbf{P}_{ss'}^\pi = \sum_a \pi(a \mid s) \mathcal{P}_{ss'}^a$ and $\mathbf{r}_s^\pi = \sum_a \pi(a \mid s) \mathcal{R}_s^a$. Because the right-hand side is linear in $\mathbf{v}_\pi$, the equation can be solved directly as $\mathbf{v}_\pi = (\mathbf{I} - \gamma \mathbf{P}^\pi)^{-1} \mathbf{r}^\pi$ at cost $\mathcal{O}(n^3)$ in the number of states. For large state spaces this is infeasible, and iterative policy evaluation is used instead.

Interpretation. The Bellman expectation equation is a self-consistency condition: the value of a state must equal the average outcome of one step under π plus the discounted value of where one lands. Every policy-evaluation algorithm in dynamic programming, Monte Carlo, and temporal-difference learning ultimately reduces to enforcing this consistency, with different methods differing only in whether and how the right-hand side is sampled or bootstrapped.

4. **Define the optimal value functions v_* and q_* and derive the Bellman optimality equations. Discuss the relationship between v_* and q_* , the non-linearity introduced by the max operator, and how the optimal policy is read off from v_* or q_* .**

Definition. The optimal state-value function is $v_*(s) = \max_\pi v_\pi(s)$, the highest expected return achievable from s over all policies. The optimal action-value function is $q_*(s, a) =$

$\max_{\pi} q_{\pi}(s, a)$, the highest expected return achievable by taking a first in s and then following the best policy thereafter. A policy π_* is optimal if and only if it achieves v_* (equivalently q_*) at every state; for finite MDPs at least one deterministic optimal policy always exists.

Derivation of the Bellman optimality equation for v_ .* Since an optimal policy must select the best one-step action from every state,

$$\begin{aligned} v_*(s) &= \max_a q_*(s, a) \\ &= \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')]. \end{aligned}$$

The first line says the best action from s is the one with the largest action value; the second substitutes the one-step lookahead expression for q_* .

Bellman optimality equation for q_ .* Applying the same argument to the action-value function,

$$q_*(s, a) = \sum_{s', r} p(s', r \mid s, a) [r + \gamma \max_{a'} q_*(s', a')].$$

The maximum now appears inside the bracket because the agent will, after the first step, again choose the best action at the successor state.

Non-linearity. Unlike the Bellman expectation equation, the right-hand side here contains $\max_a$, which is piecewise linear but not linear in v_* . Consequently, the equation cannot be solved by a single matrix inversion. Instead, the right-hand side is applied as an update rule, sweeping over the states until the values stop changing; this is exactly the value iteration algorithm of the last question.

Reading off the optimal policy. Once v_* or q_* is known, a deterministic optimal policy is obtained greedily:

- from v_* : $\pi_*(s) \in \arg \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_*(s')]$, which requires the model;
- from q_* : $\pi_*(s) \in \arg \max_a q_*(s, a)$, which requires no model.

The action-value representation is therefore the more convenient one in model-free settings, since the policy can be extracted by inspection.

Practical aspects. The Bellman optimality equation has a unique solution under $\gamma < 1$ and bounded rewards. Every greedy policy with respect to that solution is optimal, and at every state the set of arg-max actions can be used interchangeably. Ties produce equally good deterministic optimal policies, of which there may be many; any one of them attains v_* from every starting state. In implementation, the optimality equation is most often used in three places: as the update rule inside value iteration; as the target of the improvement step inside policy iteration; and as the stopping condition (the policy is optimal precisely when greedy improvement makes no change).

Connection to DP. The optimality equation is the target that every dynamic programming method aims to satisfy. Policy iteration approaches it by alternating between exact policy evaluation and greedy improvement, while value iteration approaches it by iterating the optimality operator directly. The remaining model-free methods of later units (Monte Carlo, temporal-difference) all preserve the same target while estimating the expectations from sampled experience.

5. Describe iterative policy evaluation in full. Provide the update rule, the pseudocode, the convergence argument, and the role of the tolerance parameter θ .

Definition. Policy evaluation, also called the prediction problem, is the task of computing the

state-value function v_π of a given policy π when the model $p(s', r | s, a)$ is known. The iterative algorithm starts from an arbitrary V_0 (with $V_0(\text{terminal}) = 0$) and applies the Bellman expectation backup repeatedly until the values stop changing.

Update rule. For each state s the update is

$$V_{k+1}(s) \leftarrow \sum_a \pi(a | s) \sum_{s', r} p(s', r | s, a) [r + \gamma V_k(s')].$$

The right-hand side is exactly the Bellman expectation backup, treating V_k as the current estimate of v_π and producing the next estimate V_{k+1} . When implemented with two arrays (one for V_k and one for V_{k+1}), the sweep is called *synchronous*; an in-place version (Gauss–Seidel style) overwrites $V_k(s)$ immediately and typically converges faster in practice.

Pseudocode.

Iterative Policy Evaluation

```

Input: policy pi, model p, discount gamma, tolerance theta.
Init: V(s) = 0 for all s; V(terminal) = 0.

Repeat:
  Delta = 0
  For each state s in S:
    v = V(s)
    V(s) = sum_a pi(a|s) * sum_{s',r} p(s',r|s,a) [ r + gamma * V(s') ]
    Delta = max( Delta, | v - V(s) | )
Until Delta < theta

Return V approximately equal to v_pi.
```

Role of the tolerance θ . The loop is stopped when the largest single-state change $\Delta = \max_s |V_{k+1}(s) - V_k(s)|$ drops below a small tolerance θ . Smaller θ gives more accurate evaluation at additional cost; in practice exact convergence is unnecessary because the values are only an intermediate quantity used inside policy iteration or for greedy improvement, both of which only need V accurate enough that the resulting greedy actions are correct.

Implementation notes.

- *In-place updates* (Gauss–Seidel style) overwrite $V(s)$ immediately within a sweep, so later states in the same sweep already see the updated values. This is usually faster than maintaining two arrays.
- Each sweep costs $\mathcal{O}(|\mathcal{S}|^2|\mathcal{A}|)$ for a fully connected MDP, since every state visits every action and every possible successor.
- Smaller γ tends to require fewer sweeps but emphasises near-term rewards; the modeller picks γ for problem reasons, not for speed.
- When evaluation is used inside policy iteration, warm-starting from the previous v_π is much faster than restarting from zero.

Limitations. The algorithm requires complete knowledge of the model $p(s', r | s, a)$, which is often unavailable in practice. The Monte Carlo and temporal-difference methods of Units III and IV are model-free alternatives that estimate the same expectations from sampled experience.

6. **State and prove the Policy Improvement Theorem. Use it to derive the greedy policy construction and explain when policy improvement halts.**

Statement. Let π and π' be two deterministic policies such that for every state s ,

$$q_\pi(s, \pi'(s)) \geq v_\pi(s).$$

Then π' is at least as good as π from every starting state: $v_{\pi'}(s) \geq v_\pi(s)$ for all s . If the inequality is strict at any state, then $v_{\pi'}(s) > v_\pi(s)$ at that state.

Proof sketch. Starting from the assumption and repeatedly expanding the action-value into a one-step lookahead followed by the value at the next state,

$$\begin{aligned} v_\pi(s) &\leq q_\pi(s, \pi'(s)) \\ &= \mathbb{E}_{\pi'}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \\ &\leq \mathbb{E}_{\pi'}[R_{t+1} + \gamma q_\pi(S_{t+1}, \pi'(S_{t+1})) \mid S_t = s] \\ &= \mathbb{E}_{\pi'}[R_{t+1} + \gamma R_{t+2} + \gamma^2 v_\pi(S_{t+2}) \mid S_t = s]. \end{aligned}$$

The pattern continues: each pass replaces v_π at the current frontier with q_π using the assumption, then expands one more step. After k passes,

$$v_\pi(s) \leq \mathbb{E}_{\pi'} \left[\sum_{j=1}^k \gamma^{j-1} R_{t+j} + \gamma^k v_\pi(S_{t+k}) \mid S_t = s \right].$$

The remainder term $\gamma^k v_\pi(S_{t+k})$ tends to zero as $k \rightarrow \infty$ under $\gamma < 1$ and bounded rewards, so the right-hand side converges to $\mathbb{E}_{\pi'}[G_t \mid S_t = s] = v_{\pi'}(s)$. This gives $v_\pi(s) \leq v_{\pi'}(s)$, as required.

Greedy policy construction. The natural way to satisfy the theorem's assumption is to choose π' greedy with respect to v_π :

$$\pi'(s) \in \arg \max_a q_\pi(s, a) = \arg \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_\pi(s')].$$

Because $\max_a q_\pi(s, a) \geq \sum_a \pi(a \mid s) q_\pi(s, a) = v_\pi(s)$, the greedy policy always satisfies the hypothesis of the theorem, so policy improvement is guaranteed: $v_{\pi'}(s) \geq v_\pi(s)$ for every state.

When improvement halts. Suppose the greedy policy π' is equal in value to π , so $v_{\pi'} = v_\pi$. Substituting into the greedy definition gives

$$v_{\pi'}(s) = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_{\pi'}(s')],$$

which is precisely the Bellman optimality equation. The unique solution is v_* , so $\pi' = \pi_*$. In other words, the only way for policy improvement to stop strictly improving is for the current policy to be already optimal.

Consequences. Two important properties follow.

- Each policy in the improvement sequence is strictly better than the previous one unless it is already optimal.
- A finite MDP has only finitely many deterministic policies, so the sequence terminates in a bounded number of steps. The bound is loose — typically only a handful of improvements is needed in practice.

Stochastic policies. The theorem extends to stochastic policies by replacing $q_\pi(s, \pi'(s))$ with $\sum_a \pi'(a \mid s) q_\pi(s, a)$. The greedy improvement step then collapses the stochastic policy to a deterministic one, since every probability mass is placed on an arg-max action.

Role in dynamic programming. The theorem is the engine that drives Policy Iteration: it certifies that the greedy step after each evaluation phase cannot worsen the policy and that the only stopping point of the procedure is the optimal policy.

7. Present the Policy Iteration algorithm in detail. Include the update rules, the pseudocode, the convergence argument, and a comparison with Generalised Policy Iteration (GPI).

Definition. Policy Iteration alternates two phases until the policy stops changing. The evaluation phase computes v_π exactly for the current π by iterating the Bellman expectation backup. The improvement phase constructs a new policy π' greedily from v_π . The two phases together yield the chain

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \dots \xrightarrow{I} \pi_*.$$

Pseudocode.

Policy Iteration

```

Input: model p, discount gamma, tolerance theta.
Init: V(s) arbitrary; V(terminal) = 0; pi(s) arbitrary for all s.

Repeat:
  -- Policy Evaluation
  Repeat:
    Delta = 0
    For each s in S:
      v = V(s)
      V(s) = sum_{s',r} p(s',r|s,pi(s)) [ r + gamma * V(s') ]
      Delta = max( Delta, | v - V(s) | )
  Until Delta < theta.

  -- Policy Improvement
  stable = true
  For each s in S:
    old = pi(s)
    pi(s) = argmax_a sum_{s',r} p(s',r|s,a) [ r + gamma * V(s') ]
    If old != pi(s):
      stable = false
  Until stable = true.

Return pi approximately equal to pi_*, V approximately equal to v_*.

```

Why it terminates. Each greedy improvement produces a policy that is at least as good as the previous one (by the Policy Improvement Theorem), and a finite MDP has only finitely many deterministic policies. A strictly improving sequence cannot revisit a policy and therefore must stop. The only stable point is a policy whose greedy update makes no change, which by the optimality equation is the optimal policy itself.

Practical behaviour. Empirically, the algorithm converges in only a handful of outer iterations even for problems with hundreds of states (the classical Jack's-Car-Rental example with 441 states converges in roughly four to five outer steps). Two implementation refinements are common: warm-starting each evaluation phase from the previous value function rather than from scratch, which makes the inner loop very fast; and stopping evaluation early as soon as $\Delta < \theta$ for a small tolerance, since exact convergence is never needed for the greedy step to choose the correct action.

Strengths.

- Strong convergence guarantee in a finite number of steps.
- Few outer iterations are needed; each improvement is a single sweep.

- Easy to extend to action values: replacing $V(s)$ with $Q(s, a)$ removes the need for the model at the improvement step.

Limitations.

- Each evaluation phase is itself an iterative loop, which may be expensive when the inner-loop tolerance is small.
- Like all dynamic programming methods, it requires the complete model $p(s', r | s, a)$.
- Synchronous sweeps over all states do not scale to very large state spaces; the asynchronous variants discussed below are preferred there.

Generalised Policy Iteration. Generalised Policy Iteration (GPI) is the umbrella idea that covers any algorithm that alternates between an evaluation step (making V more consistent with π) and an improvement step (making π more greedy with respect to V), with no restriction on how much each step is allowed to do. Policy Iteration corresponds to full evaluation followed by full improvement. Value Iteration (next question) reduces evaluation to a single sweep before improving. Asynchronous DP performs both steps one state at a time. The model-free methods of later units also fall under GPI, differing only in how the evaluation step estimates the Bellman backup from samples.

8. Develop the Value Iteration algorithm. Provide the update rule, the pseudocode, the contraction argument, and compare it with Policy Iteration. Briefly mention asynchronous dynamic programming as an extension.

Motivation. The gridworld example shows that the greedy policy can become optimal long before policy evaluation has fully converged. This suggests that a single Bellman expectation sweep, followed immediately by a greedy improvement, is enough; collapsing the two steps into one gives Value Iteration.

Update rule. Value Iteration applies the Bellman optimality backup at every state on every sweep:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V_k(s')].$$

The maximum over actions implicitly performs the improvement step, while the lookahead over successor states performs one step of evaluation. There is no separate policy variable during the iteration; the policy is extracted only at the end.

Pseudocode.

Value Iteration

Input: model p , discount γ , tolerance θ .

Init: $V(s)$ arbitrary; $V(\text{terminal}) = 0$.

Repeat:

$\Delta = 0$

 For each s in S :

$v = V(s)$

$V(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$

$\Delta = \max(\Delta, |v - V(s)|)$

Until $\Delta < \theta$.

Output policy:

$\pi(s) = \operatorname{argmax}_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$

Stopping criterion. The loop is stopped when the largest change $\Delta = \max_s |V_{k+1}(s) - V_k(s)|$ across a full sweep falls below a small tolerance θ . In practice this criterion is generous: the greedy policy extracted from a roughly correct V is often already optimal because small errors in the values rarely change which action is the arg-max.

Comparison with Policy Iteration.

- Per outer step, Value Iteration is cheaper: a single Bellman optimality sweep instead of a full evaluation loop.
- Per outer step, Policy Iteration is more powerful: each improvement is guaranteed to reach the value of the next policy, which often results in only five to twenty outer iterations being needed.
- Value Iteration generally requires more outer sweeps because each sweep makes only a small amount of progress.
- Total computation is often comparable, with the better choice depending on the problem and the cost of the inner evaluation loop. Modified policy iteration (a hybrid that runs a small fixed number of evaluation sweeps before each improvement) interpolates between the two.

Policy extraction. Unlike Policy Iteration, no explicit policy is maintained during Value Iteration. Once the value function has converged, a deterministic optimal policy is read off in a single greedy sweep:

$$\pi_*(s) \in \arg \max_a \sum_{s',r} p(s',r | s,a) [r + \gamma V(s')].$$

If two actions tie at a state, either may be chosen; the resulting policy is still optimal.

Asynchronous dynamic programming. A synchronous sweep updates every state once per outer iteration; for very large state spaces this is wasteful because many states have already converged. Asynchronous DP relaxes the rule and allows the values of individual states to be updated in any order and at any rate, provided every state is visited infinitely often. Two variants are common:

- *Asynchronous value iteration* picks one state at a time and applies the Bellman optimality backup to it alone;
- *Real-time DP* updates only the states the agent actually visits during interaction with the environment, focusing computation on the states that matter for the current policy.

Asynchronous methods do not reduce the total work, but they redirect it towards the states that influence the current policy first, which is invaluable when $|\mathcal{S}|$ is large.

Bridge. Both Value Iteration and Policy Iteration are instances of Generalised Policy Iteration — the unifying view that all reinforcement-learning algorithms alternate evaluation and improvement. The algorithms of later units (Monte Carlo, temporal-difference, Q-learning) preserve this skeleton and only change how the evaluation step is realised: they replace the model-based Bellman backup with a sample-based estimate drawn from interaction with the environment.